

OpenClaw Plugin Function Introduction

OpenClaw Plugin Function Introduction

- Quick Start
- Available Plugins (Official)
- Runtime Utilities
- Discovery and Priority
 - Package Collections
 - Channel Directory Metadata
- Plugin ID
- Configuration
- Plugin Slots (Exclusive Categories)
- Control Interface (schema + hints)
- CLI
- Plugin API (Overview)
- Plugin Hooks
 - Example
- Provider Plugins (Model Authentication)
 - Registering Message Channels
 - Writing a New Message Channel (Step-by-Step Guide)
 - Agent Tools
 - Register Gateway RPC Methods
 - Register CLI Commands
 - Register Auto-Reply Commands
 - Register Background Services
- Naming Conventions
- Skills
- Distribution (npm)
- Example Plugin: Voice Call
- Security Notes
- Testing Plugins

Quick Start

A plugin is just a **small code module** that extends OpenClaw with additional functionality (commands, tools, and Gateway RPC). Most of the time, you will use plugins when you want a feature that hasn't been built into the core OpenClaw yet (or you want to keep optional features out of the main installation). Quick path:

1. View loaded content:

```
openclaw plugins list
```

2. Install official plugins (e.g., Voice Call):

```
openclaw plugins install @openclaw/voice-call
```

3. Restart Gateway and configure under `plugins.entries.<id>.config`.

Available Plugins (Official)

- Starting from 2026.1.15, Microsoft Teams is only available as a plugin; if using Teams, install `@openclaw/msteams`.
- Memory (Core) — Bundled memory search plugin (enabled by default via `plugins.slots.memory`)
- Memory (LanceDB) — Bundled long-term memory plugin (automatic recall/capture; set `plugins.slots.memory = "memory-lancedb"`)
- [Voice Call](#) — `@openclaw/voice-call`
- [Zalo Personal](#) — `@openclaw/zalouser`
- [Matrix](#) — `@openclaw/matrix`
- [Nostr](#) — `@openclaw/nostr`
- [Zalo](#) — `@openclaw/zalo`
- [Microsoft Teams](#) — `@openclaw/msteams`
- Google Antigravity OAuth (Provider Auth) — Bundled as `google-antigravity-auth` (disabled by default)
- Gemini CLI OAuth (Provider Auth) — Bundled as `google-gemini-cli-auth` (disabled by default)
- Qwen OAuth (Provider Auth) — Bundled as `qwen-portal-auth` (disabled by default)
- Copilot Proxy (Provider Auth) — Local VS Code Copilot Proxy bridge; different from built-in `github-copilot` device login (bundled, disabled by default)

OpenClaw plugins are **TypeScript modules** loaded at runtime via jiti. **Configuration validation does not execute plugin code**; it uses the plugin manifest and JSON Schema. See [Plugin Manifest](#). Plugins can register:

- Gateway RPC methods
- Gateway HTTP handlers
- Agent tools
- CLI commands
- Background services
- Optional configuration validation
- **Skills** (by listing `skills` directory in the plugin manifest)
- **Auto-reply commands** (execute without invoking the AI agent)

Plugins run in the **same process** as the Gateway, so treat them as trusted code. Tool writing guide: [Plugin Agent Tools](#).

Runtime Utilities

Plugins can access selected core utilities through `api.runtime`. For telephony TTS:

```
const result = await api.runtime.tts.textToSpeechTelephony({
  text: "Hello from OpenClaw",
  cfg: api.config,
});
```

Notes:

- Uses core `messages.tts` configuration (OpenAI or ElevenLabs).
- Returns PCM audio buffer + sample rate. Plugins must resample/encode for their provider.
- Edge TTS does not support telephony.

Discovery and Priority

OpenClaw scans in order:

1. Config paths

- `plugins.load.paths` (file or directory)

2. Workspace extensions

- `<workspace>/openclaw/extensions/*.ts`
- `<workspace>/openclaw/extensions/*/index.ts`

3. Global extensions

- `~/.openclaw/extensions/*.ts`
- `~/.openclaw/extensions/*/index.ts`

4. Bundled extensions (released with OpenClaw, **disabled by default**)

- `<openclaw>/extensions/*`

Bundled plugins must be explicitly enabled via `plugins.entries.<id>.enabled` or `openclaw plugins enable <id>`. Installed plugins are enabled by default but can be disabled the same way. Each plugin must contain an `openclaw.plugin.json` file in its root directory. If the path points to a file, the plugin root is that file's directory, which must contain the manifest. If multiple plugins resolve to the same id, the first match in the above order wins, and lower priority copies are ignored.

Package Collections

Plugin directories can contain a `package.json` with `openclaw.extensions`:

```
{
  "name": "my-pack",
  "openclaw": {
    "extensions": [".src/safety.ts", ".src/tools.ts"]
  }
}
```

Each entry becomes a plugin. If a package lists multiple extensions, the plugin id becomes `name/<fileBase>`. If your plugin imports npm dependencies, install them in that directory so `node_modules` is available (`npm install` / `pnpm install`).

Channel Directory Metadata

Channel plugins can broadcast onboarding metadata via `openclaw.channel` and installation hints via `openclaw.install`. This keeps the core directory data-free. Example:

```
{
  "name": "@openclaw/nextcloud-talk",
  "openclaw": {
    "extensions": [".index.ts"],
    "channel": {
      "id": "nextcloud-talk",
      "label": "Nextcloud Talk",
      "selectionLabel": "Nextcloud Talk (self-hosted)",

```

```

    "docsPath": "/channels/nextcloud-talk",
    "docsLabel": "nextcloud-talk",
    "blurb": "Self-hosted chat via Nextcloud Talk webhook bots.",
    "order": 65,
    "aliases": ["nc-talk", "nc"]
  },
  "install": {
    "npmSpec": "@openclaw/nextcloud-talk",
    "localPath": "extensions/nextcloud-talk",
    "defaultChoice": "npm"
  }
}

```

OpenClaw can also merge **external channel directories** (e.g., MPM registry exports). Place JSON files in one of:

- `~/.openclaw/mpm/plugins.json`
- `~/.openclaw/mpm/catalog.json`
- `~/.openclaw/plugins/catalog.json`

Or point `OPENCLAW_PLUGIN_CATALOG_PATHS` (or `OPENCLAW_MPM_CATALOG_PATHS`) to one or more JSON files (comma/semicolon/ `PATH` separated). Each file should contain `{ "entries": [ { "name": "@scope/pkg", "openclaw": { "channel": {...}, "install": {...} } } ] }`.

Plugin ID

Default plugin ids:

- Package collection: `name` from `package.json`
- Standalone file: file base name (`~/.../voice-call.ts` → `voice-call`)

If a plugin exports `id`, OpenClaw uses it but warns when it doesn't match the configured id.

Configuration

```

{
  plugins: {
    enabled: true,
    allow: ["voice-call"],
    deny: ["untrusted-plugin"],
    load: { paths: ["~/Projects/oss/voice-call-extension"] },
    entries: {
      "voice-call": { enabled: true, config: { provider: "twilio" } },
    },
  },
}

```

Fields:

- `enabled`: Main switch (default: true)
- `allow`: Allow list (optional)
- `deny`: Deny list (optional; deny takes precedence)
- `load.paths`: Additional plugin files/directories
- `entries.<id>`: Switch + config per plugin

Configuration changes **require Gateway restart**. Validation rules (strict):

- Unknown plugin ids in `entries`, `allow`, `deny`, or `slots` are **errors**.
- Unknown `channels.<id>` keys are **errors** unless the plugin manifest declares the channel id.
- Plugin configuration is validated using JSON Schema embedded in `openclaw.plugin.json` (`configSchema`).
- If a plugin is disabled, its configuration is preserved and a **warning** is issued.

Plugin Slots (Exclusive Categories)

Some plugin categories are **exclusive** (only one active at a time). Use `plugins.slots` to select which plugin owns the slot:

```
{
  plugins: {
    slots: {
      memory: "memory-core", // or "none" to disable memory plugins
    },
  },
}
```

If multiple plugins declare `kind: "memory"`, only the selected one loads. Others are disabled with diagnostics.

Control Interface (schema + hints)

The control interface uses `config.schema` (JSON Schema + `uiHints`) to render better forms. OpenClaw enhances `uiHints` at runtime based on discovered plugins:

- Add per-plugin labels for `plugins.entries.<id>` / `.enabled` / `.config`
- Merge optional plugin-provided config field hints at: `plugins.entries.<id>.config.<field>`

If you want plugin config fields to show nice labels/placeholders (and mark keys as sensitive), provide `uiHints` and JSON Schema in the plugin manifest. Example:

```
{
  "id": "my-plugin",
  "configSchema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "apiKey": { "type": "string" },
      "region": { "type": "string" }
    }
  },
  "uiHints": {
    "apiKey": { "label": "API Key", "sensitive": true },
    "region": { "label": "Region", "placeholder": "us-east-1" }
  }
}
```

CLI

```

openclaw plugins list
openclaw plugins info <id>
openclaw plugins install <path> # copy a local file/dir into
~/.openclaw/extensions/<id>
openclaw plugins install ./extensions/voice-call # relative path ok
openclaw plugins install ./plugin.tgz # install from a local tarball
openclaw plugins install ./plugin.zip # install from a local zip
openclaw plugins install -l ./extensions/voice-call # link (no copy) for dev
openclaw plugins install @openclaw/voice-call # install from npm
openclaw plugins update <id>
openclaw plugins update --all
openclaw plugins enable <id>
openclaw plugins disable <id>
openclaw plugins doctor

```

`plugins update` only works for npm installations tracked under `plugins.installs`. Plugins can also register their own top-level commands (e.g., `openclaw voicecall`).

Plugin API (Overview)

Plugins export one of:

- Function: `(api) => { ... }`
- Object: `{ id, name, configSchema, register(api) { ... } }`

Plugin Hooks

Plugins can ship hooks and register them at runtime. This allows plugins to bundle event-driven automation without a separate hook package.

Example

```

import { registerPluginHooksFromDir } from "openclaw/plugin-sdk";

export default function register(api) {
  registerPluginHooksFromDir(api, "./hooks");
}

```

Notes:

- Hooks directory follows normal hook structure (`HOOK.md` + `handler.ts`).
- Hook qualification rules still apply (OS/binary/environment/config requirements).
- Plugin-managed hooks appear as `plugin:<id>` in `openclaw hooks list`.
- You cannot enable/disable plugin-managed hooks via `openclaw hooks`; enable/disable the plugin instead.

Provider Plugins (Model Authentication)

Plugins can register **model provider authentication** flows so users can run OAuth or API key setup within OpenClaw (no external scripts needed). Register providers via `api.registerProvider(...)`. Each provider exposes one or more authentication methods (OAuth, API key, device code, etc.). These methods drive:

- `openclaw models auth login --provider <id> [--method <id>]`

Example:

```
api.registerProvider({
  id: "acme",
  label: "AcmeAI",
  auth: [
    {
      id: "oauth",
      label: "OAuth",
      kind: "oauth",
      run: async (ctx) => {
        // Run OAuth flow and return auth profiles.
        return {
          profiles: [
            {
              profileId: "acme:default",
              credential: {
                type: "oauth",
                provider: "acme",
                access: "...",
                refresh: "...",
                expires: Date.now() + 3600 * 1000,
              },
            },
          ],
          defaultModel: "acme/opus-1",
        };
      },
    ],
  ],
});
```

Notes:

- `run` receives `ProviderAuthContext` with `prompter`, `runtime`, `openUrl`, and `oauth.createVpsAwareHandlers` utilities.
- Return `configPatch` when adding default models or provider configuration.
- Return `defaultModel` so `--set-default` can update agent defaults.

Registering Message Channels

Plugins can register **channel plugins** that behave like built-in channels (WhatsApp, Telegram, etc.). Channel configuration lives under `channels.<id>` and is validated by your channel plugin code.

```
const myChannel = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "demo channel plugin.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
};
```

```

config: {
  listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ?? {}),
  resolveAccount: (cfg, accountId) =>
    cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
      accountId,
    },
},
outbound: {
  deliveryMode: "direct",
  sendText: async () => ({ ok: true }),
},
};

export default function (api) {
  api.registerChannel({ plugin: myChannel });
}

```

Notes:

- Put configuration under `channels.<id>` (not `plugins.entries`).
- `meta.label` is used for labels in CLI/UI lists.
- `meta.aliases` adds alternate ids for normalization and CLI input.
- `meta.preferOver` lists channel ids to skip auto-enabling when both are configured.
- `meta.detailLabel` and `meta.systemImage` let the UI show richer channel labels/icons.

Writing a New Message Channel (Step-by-Step Guide)

Use this approach when you want a **new chat interface** ("message channel") rather than a model provider. Model provider documentation is under `/providers/*`.

1. Choose id + config structure

- All channel configuration lives under `channels.<id>`.
- For multi-account setups, prefer `channels.<id>.accounts.<accountId>`.

2. Define channel metadata

- `meta.label`, `meta.selectionLabel`, `meta.docspath`, `meta.blurb` control CLI/UI listings.
- `meta.docspath` should point to a doc page like `/channels/<id>`.
- `meta.preferOver` lets a plugin replace another channel (auto-enable prefers it).
- `meta.detailLabel` and `meta.systemImage` are used by the UI for detail text/icon.

3. Implement required adapters

- `config.listAccountIds` + `config.resolveAccount`
- `capabilities` (chat types, media, threads, etc.)
- `outbound.deliveryMode` + `outbound.sendText` (for basic sending)

4. Add optional adapters as needed

- `setup` (wizard), `security` (DM policy), `status` (health/diagnostics)
- `gateway` (start/stop/login), `mentions`, `threading`, `streaming`
- `actions` (message operations), `commands` (native command behavior)

5. Register the channel in the plugin

- `api.registerChannel({ plugin })`

Minimal config example:

```
{
  channels: {
    acmechat: {
      accounts: {
        default: { token: "ACME_TOKEN", enabled: true },
      },
    },
  },
}
```

Minimal channel plugin (outbound only):

```
const plugin = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "AcmeChat messaging channel.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
  config: {
    listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ?? {}),
    resolveAccount: (cfg, accountId) =>
      cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
        accountId,
      },
  },
  outbound: {
    deliveryMode: "direct",
    sendText: async ({ text }) => {
      // deliver `text` to your channel here
      return { ok: true };
    },
  },
};

export default function (api) {
  api.registerChannel({ plugin });
}
```

Load the plugin (extensions directory or `plugins.load.paths`), restart Gateway, then configure `channels.<id>` in config.

Agent Tools

See dedicated guide: [Plugin Agent Tools](#).

Register Gateway RPC Methods

```
export default function (api) {
  api.registerGatewayMethod("myplugin.status", ({ respond }) => {
    respond(true, { ok: true });
  });
}
```

Register CLI Commands

```
export default function (api) {
  api.registerCli(
    ({ program }) => {
      program.command("mycmd").action(() => {
        console.log("Hello");
      });
    },
    { commands: ["mycmd"] },
  );
}
```

Register Auto-Reply Commands

Plugins can register custom slash commands that **execute without invoking the AI agent**. This is useful for toggle commands, status checks, or quick operations that don't need LLM processing.

```
export default function (api) {
  api.registerCommand({
    name: "mystatus",
    description: "show plugin status",
    handler: (ctx) => ({
      text: `Plugin is running! Channel: ${ctx.channel}`,
    }),
  });
}
```

Command handler context:

- `senderId`: ID of the sender (if available)
- `channel`: Channel where the command was sent
- `isAuthorizedSender`: Whether sender is an authorized user
- `args`: Arguments passed after the command (if `acceptsArgs: true`)
- `commandBody`: Complete command text
- `config`: Current OpenClaw configuration

Command options:

- `name`: Command name (without leading `/`)
- `description`: Help text shown in command list
- `acceptsArgs`: Whether command accepts arguments (default: false). If false and arguments are provided, the command won't match and the message goes to other handlers
- `requireAuth`: Whether authorized sender is required (default: true)
- `handler`: Function returning `{ text: string }` (can be async)

Example with auth and args:

```
api.registerCommand({
  name: "setmode",
  description: "Set plugin mode",
  acceptsArgs: true,
  requireAuth: true,
  handler: async (ctx) => {
    const mode = ctx.args?.trim() || "default";
    await saveMode(mode);
    return { text: `Mode set to: ${mode}` };
  },
});
```

Notes:

- Plugin commands are processed before built-in commands and AI agent
- Commands are registered globally and apply to all channels
- Command names are case-insensitive (`/MyStatus` matches `/mystatus`)
- Command names must start with a letter and can only contain letters, numbers, hyphens, and underscores
- Reserved command names (like `help`, `status`, `reset`, etc.) cannot be overridden by plugins
- Duplicate command registration across plugins will fail with diagnostic error

Register Background Services

```
export default function (api) {
  api.registerService({
    id: "my-service",
    start: () => api.logger.info("ready"),
    stop: () => api.logger.info("bye"),
  });
}
```

Naming Conventions

- Gateway methods: `pluginId.action` (e.g., `voicecall.status`)
- Tools: `snake_case` (e.g., `voice_call`)
- CLI commands: kebab or camel, but avoid conflicts with core commands

Skills

Plugins can ship Skills in their repo (`skills/<name>/SKILL.md`). Enable via `plugins.entries.<id>.enabled` (or other config gating), and ensure it exists in your workspace/hosted Skills location.

Distribution (npm)

Recommended packaging:

- Main package: `openclaw` (this repo)
- Plugins: Separate npm packages under `@openclaw/*` (e.g., `@openclaw/voice-call`)

Publish contract:

- Plugin `package.json` must contain `openclaw.extensions` with one or more entry files.
- Entry files can be `.js` or `.ts` (jiti loads TS at runtime).
- `openclaw plugins install <npm-spec>` uses `npm pack`, extracts to `~/.openclaw/extensions/<id>/`, and enables it in config.
- Config key stability: Scoped packages are normalized to unscoped ids for `plugins.entries.*`.

Example Plugin: Voice Call

This repo contains a voice call plugin (Twilio or log fallback):

- Source: `extensions/voice-call`
- Skills: `skills/voice-call`
- CLI: `openclaw voicecall start|status`
- Tools: `voice_call`
- RPC: `voicecall.start`, `voicecall.status`
- Config (twilio): `provider: "twilio"` + `twilio.accountSid/authToken/from` (optional `statusCallbackUrl`, `twiMLUrl`)
- Config (dev): `provider: "log"` (no network)

See [Voice Call](#) and `extensions/voice-call/README.md` for setup and usage.

Security Notes

Plugins run in the same process as the Gateway. Treat them as trusted code:

- Only install plugins you trust.
- Prefer using `plugins.allow` allow list.
- Restart Gateway after changes.

Testing Plugins

Plugins can (and should) ship with tests:

- In-repo plugins can keep Vitest tests under `src/**` (e.g., `src/plugins/voice-call.plugin.test.ts`).
- Individually released plugins should run their own CI (lint/build/test) and verify `openclaw.extensions` points to built entry points (`dist/index.js`).

For more content about plugins, please refer to: [Plugins - OpenClaw](#)